



Computer vision aided floorplan analysis

Sonia Khalili

1409747, Khalili@student.unimelb.edu.au

Louisa Markby

1270733, lmarkby@student.unimelb.edu.au

Zeqi Yu

1118869, zeqiy@student.unimelb.edu.au

Executive Summary:

This project developed a computer-visioned pipeline for automated floor plan analysis to estimate interior space, objects occupied area, text occupied area, and free usable space from architectural floor plan images. The project was motivated by the limitations of traditional manual floor plan analysis methods, which are often time-consuming, labour-intensive, and prone to human estimation errors. Therefore, the proposed system combines traditional image processing, deep learning-based object detection, and OCR-based text detection into a three-branch framework to improve the accuracy of free space percentage calculation.

The first branch uses OpenCV image processing method to extract the interior area of a floor plan through thresholding, morphological operations, and flood fill analysis. The second branch applies a Mask R-CNN instance segmentation model to detect areas occupied by objects such as doors, showers, sinks, toilets and other furniture at the pixel level. The third branch employs an EasyOCR-based model to detect the text area and estimate the pixel area occupied by the texts, and then the region can be added back into the free space calculation.

Experimental results showed that the proposed pipeline successfully extract the interior area, areas occupied by objects, and the area occupied by texts across multiple floor plan layouts. Quantitative evaluation also demonstrated strong segmentation performance with IoU values ranging from 0.63 to 0.80, and the F1-scores was up to 0.88. The project concludes that the proposed three-branch pipeline provides a practical, scalable, and sustainable framework for automated floor plan analysis and free-space estimation.

Table of Content

1. Introduction.....	Error! Bookmark not defined.
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Objectives	1
2. Literature Review.....	1
2.1 Convolutional Neural Networks for Object Detection	2
2.3 Mask R-CNN for Instance Segmentation	2
2.4 Precision Recall in Two-Class Detection.....	2
3. Methodology	4
3.1 System Overview.....	4
3.1.1 Branch A: Image Processing (Interior Area Extraction)	5
3.1.2 Branch B: Mask R-CNN Object Detection (Occupied Area)	5
3.1.3 Branch C: OCR-Based Text Detection (Text Area Estimation).....	6
3.1.4 The Implementation Environment	7
4. Results and Analysis	7
4.1 Experimental Setup.....	7
4.2 Interior Area Extraction Results (Branch A)	7
4.3 Object Detection Results (Branch B)	9
4.4 Text Detection Results (Branch C)	10
4.5 Free Space Estimation Results without Text Pixels VS Free Space Estimation Results with Text Pixels	12
4.6 Quantitative Evaluation	13
5. Discussion.....	14
5.1 Interpretation of Image 1's Performance	14
5.2 Limitations of the Pipeline	14
5.3 Recommendations for Improvement	15
6. Conclusion	16
Acknowledgements.....	20
Bibliography (APA).....	18

Table of Figures

Figure 2.1 Precision Recall Curves for Different IoUs.....	3
Figure 2.2 Precision Recall Curve for Faster R-CNN.....	3
Figure 2.3 Precision-Recall and F1-Confidence curves for Yolov8 for IoU = 0.....	4
Figure 3.1 Final Pipeline.....	4
Figure 4.1 Interior Area of Image 1	8
Figure 4.2 Interior Area of Image 2	8
Figure 4.3 Interior Area of Image 3	9
Figure 4.4 Object Detection in Image 1	9
Figure 4.5 Object Detection in Image 2	10
Figure 4.6 Object Detection in Image 3	10
Figure 4.7 Text detection in Image 1	11
Figure 4.8 Text Detection in Image 2.....	11
Figure 4.9 Text Detection in Image 3.....	12
Figure 5.1 Alternative Pipeline	16
Figure 5.2 Alternative Branch C	16
Table 4.1 Results for Interior Area.....	7
Table 4.2 Results for Text Detection	11
Table 4.3 Free Space Results without Text Pixels	12
Table 4.4 Free Space Results with Text Pixels.....	12
Table 4.5 Quantitative Metrics.....	13

1. Introduction

1.1 Background

Telecommunication facilities, such as Telstra's cell rooms, store critical communication equipment that ensures network connectivity throughout Australia. In order to guarantee equipment performance, safety, and accessibility, these spaces must be structured efficiently. Manual inspection or CAD-based measurements, which are laborious and prone to human error, have historically been used to determine the amount of space available for new installations. Recent developments in deep learning and computer vision have made it possible to automatically analyse spatial layouts from floorplans and photos. While traditional image processing techniques may compute inner areas and define structural boundaries, techniques such as Mask R-CNN provide exact object segmentation. By combining these methods, a strong foundation for quantitative spatial analysis is produced, assisting with planning and facility management choices.

1.2 Problem Statement

Telstra has identified the need for an automated approach to assess the spatial layout of cell rooms containing communication equipment. The goal is to use visual data, such as engineering floorplan, to identify the occupied and free sections within the building. For large-scale operations, manual space measurement is ineffective and difficult to adjust to regular equipment changes. Creating a reliable, scalable pipeline that can evaluate various floorplans, precisely identify equipment and other objects, and calculate the free-space percentage is the difficult part.

This solution meets Telstra's needs by combining Mask-R-CNN-based object detection with image-processing techniques to generate accurate spatial metrics. The final system seeks to improve operational effectiveness, lessen manual labour, and facilitate data-driven equipment placement.

1.3 Objectives

The objectives of this project are defined as:

- To create and deploy a data-driven system that uses Mask R-CNN to identify objects in an engineering floorplan and calculate the area occupied by the detected objects
- to create an image processing pipeline that, after deducting occupied regions, determines the overall interior area and calculates the percentage of empty space.
- to verify the accuracy of the system using visual overlays on test images.

2. Literature Review

This chapter explores the body of research relevant to the three main project components: computer vision techniques for interior area extraction, instance segmentation utilizing Mask R-CNN for item detection in floor plan images, and OCR tool for detecting text regions.

2.1 Convolutional Neural Networks for Object Detection

The advent of deep CNNs profoundly altered object detection. A number of region-based detection designs were inspired by Krizhevsky et al. (2012), who showed that deep CNNs trained on massive picture datasets significantly outperformed manually created feature techniques. R-CNN, which utilized CNN feature extraction to selected search region recommendations, was first presented by Girshick et al. (2014). By sharing convolutional computation across proposals, Girshick (2015) enhanced this with Fast R-CNN. Ren et al. (2015) finished the progression with Faster R-CNN, substituting a learned Region Proposal Network for selective search and creating a fully end-to-end trainable detection system that went on to become the industry **standard** for two-stage detection.

Commented [SK1]: Zeqi, you can add your part for FAST R-CNN in 2.2 section before the MASK R-CNN. Because I am talking about the evolution from FAST to MASK- R-CNN

2.2 Faster R-CNN for Object Detection

Faster R-CNN is a two-stage deep learning object detection framework that improves previous R-CNN-based detectors by introducing a Region Proposal Network (RPN) for generating object candidate regions directly from convolutional feature maps (Ren et al., 2015). RPN significantly improves both computational efficiency and localization performance by sharing convolutional features with detection network, compared with earlier approaches that relied on external selective search algorithms.

The Faster R-CNN architecture performs object detection through two main branches. First, the RPN predicts candidate object regions from extracted feature maps. Second, the detected candidates are classified and refined through bounding box regression. This two-stage detection framework allows Faster R-CNN to achieve higher detection accuracy, especially for complex tasks.

2.3 Mask R-CNN for Instance Segmentation

By including a parallel mask prediction branch in addition to the classification and bounding box regression heads, He et al. (2017) expanded Faster R-CNN to instance segmentation and created Mask R-CNN. One important contribution was RoIAlign, which substituted bilinear interpolation for the quantized RoIPool operation, maintaining exact spatial correspondence between region proposals and feature maps and greatly enhancing mask accuracy for small objects, which are directly related to floor plan fixtures like sinks, toilets, and door arcs.

2.4 Precision Recall in Two-Class Detection

Recall measures the percentage of all ground-truth objects that were successfully detected, whereas precision measures the percentage of predicted detections that match actual annotated objects for a two-class detection model that distinguishes between background and a single object class (Davis & Goadrich, 2006).

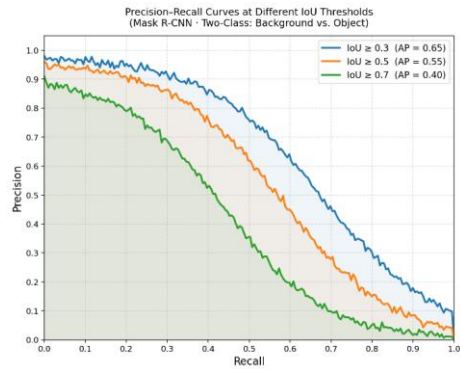


Figure 2.1 Precision Recall Curves for Different IoUs

The Mask R-CNN model's Precision-Recall curves are shown in Figure 2.1 at three IoU thresholds: ≥ 0.3 , ≥ 0.5 , and ≥ 0.7 . These thresholds provide AP values of 0.65, 0.55, and 0.40, respectively. The curves have a number of notable features. The model's highest-confidence detections are nearly always accurate, as all three curves start at or close to precision = 1.0 at relatively low recall. Precision declines gradually rather than abruptly when the confidence score threshold is gradually decreased, admitting more predictions and raising recall. This gradual trade-off is typical of a well-calibrated detector. The $\text{IoU} \geq 0.3$ curve maintains excellent precision up until around recall = 0.65 before decreasing more sharply. The $\text{IoU} \geq 0.5$ and $\text{IoU} \geq 0.7$ curves begin their descent slightly earlier, at approximately recall = 0.55 and 0.40, respectively. The expected behavior is a systematic downward shift across stricter IoU thresholds: as the localization requirement tightens, an increasing number of detections that accurately detect the presence of an object fail to meet the stricter spatial overlap criterion and are reclassified as false positives, decreasing precision at each recall level.

The Faster R-CNN model's Precision-recall curves are shown in Figure 2.2 at three IoU thresholds: ≥ 0.3 , ≥ 0.5 , and ≥ 0.7 .

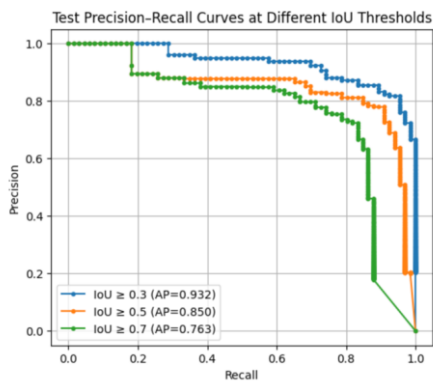


Figure 2.2 Precision Recall Curve for Faster R-CNN

According to the PV curve results shown in above figures, the Faster R-CNN model achieved higher AP values than the Mask R-CNN model at all IoU values, indicating better object detection performance on the dataset. However, Mask R-CNN is still our final choice for the branch of object detection. The primary reason is that Faster R-CNN only produces rectangular bounding-box localization, which is not accurate enough to represent the shape of irregular objects. Since the objective of the project is pixel-level occupied-area estimation rather than object detection only, precise object segmentation is more important than bounding-box detection accuracy alone. Although the Mask R-CNN achieved lower AP values, it provides instance-level area calculation. Therefore, Mask R-CNN was ultimately considered as the most suitable model for the project.

The Yolov8 model's Precision-Recall and F1-Confidence curves are shown in Figure 2.3 for all the different classes. As well as an example in Figure 2.4.

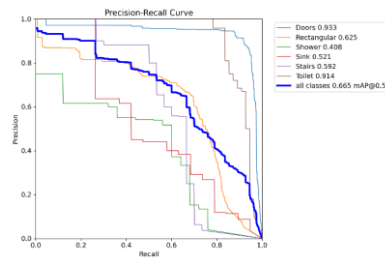


Figure 2.3 Precision-Recall and F1-Confidence curves for Yolov8 for IoU = 0

3. Methodology

3.1 System Overview

The suggested approach is a three-branch pipeline that is fully automated and is intended to calculate the occupied and free usable space in architectural floor plans. Without the need for manual measurement, the pipeline generates a quantified area breakdown expressed in pixel count from a raw floor-plan image. The general architecture is shown in Figure 3.1.

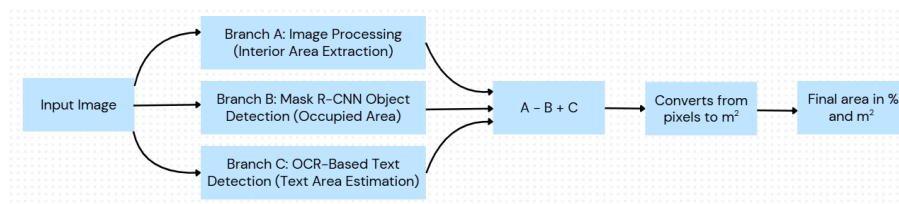


Figure 3.1 Final Pipeline

Three distinct, parallel branches of the system process the input image. One branch uses traditional computer vision techniques to extract the entire interior space of the building (excluding texts), the second branch detects interior objects and estimates their occupied area, and the third branch finds text sections inside the design. The corrected free-space estimate is obtained by combining the outputs of these branches in a final computing stage.

3.1.1 Branch A: Image Processing (Interior Area Extraction)

The first branch extracts the entire interior area of the floor layout using traditional computer vision techniques. To make further processing easier, the input image is first converted to grayscale and reduced to a single intensity channel. After that, a binary inverse threshold operation is applied at a pixel intensity value of 200. Pixels darker than this value, which correspond to wall structures, are given a value of 255 (white), while lighter pixels, which represent open room space, are given a value of 0 (black). Inverse thresholding is justified by the fact that OpenCV's morphological operations interpret white pixels as the active foreground region; rendering walls white guarantees that the structuring operations that follow will target them (Bradski, 2000).

After thresholding, minor gaps and discontinuities inside wall boundaries are sealed using a morphological closing operation that uses a 5x5 structural element with two iterations. The building boundary is then made to form a completely enclosed area by applying a morphological dilation operation with a 7x7 structural element to further thicken and link wall segments (Gonzalez & Woods, 2018). These procedures are essential to stop the subsequent flood fill from seeping into the outside through damaged wall portions.

The next step is to seed a flood fill operation at pixel location (0, 0), which in any typical floor plan image is guaranteed to lie outside the building perimeter. By filling all related outer regions with zero, this procedure essentially eliminates them from the binary image. The interior of the building is represented by the pixels that are left over after flood fill. The remaining foreground pixels are added up to determine the total interior area in pixels (Bradski, 2000).

3.1.2 Branch B: Mask R-CNN Object Detection (Occupied Area)

The second branch uses an instance segmentation model based on Mask R-CNN to detect and distinguish specific objects in the floor layout. By including a parallel branch that predicts a binary segmentation mask for every identified instance, Mask R-CNN expands the Faster R-CNN object detection framework and allows pixel-level object localization instead of only bounding-box-level approximation.

The backbone network utilized for feature extraction is a ResNet-50 architecture (He et al., 2016) combined with a Feature Pyramid Network (Lin et al., 2017). While the Feature Pyramid Network creates a multi-scale feature hierarchy that permits the detection of objects at various sizes and resolutions, which is a crucial feature for floor plan images where detected objects range from small fixtures like sinks and toilets to large structural regions, the ResNet-50 backbone extracts hierarchical feature representations from the input image through a sequence of residual convolutional blocks. The model can identify six object classes: doors, rectangular regions, showers, sinks, stairs, and toilets. It was trained on a unique dataset of floor plan photos annotated in COCO format. A bespoke Fast R-CNN classification and bounding box regression head and a Mask R-CNN segmentation head make up the model head. Both were refined on the floor plan dataset using an Adam optimizer with a learning rate scheduler during 47 training epochs.

A confidence score threshold of 0.8 is used to filter raw detections during inference in order to suppress ambiguous or low-quality detections and maintain only high-certainty predictions. After that, duplicate detections of the same object are removed using Non-Maximum Suppression (NMS) with an Intersection over Union (IoU) threshold of 0.3 (He et al., 2017). A logical union operation is then used to combine the anticipated binary masks for all retained detections, creating

Commented [SK2]: You need to add the text recognition part to the methodology. Please don't forget to use the defined styles to type.

a single composite occupied area mask where each pixel is tallied no more than once, independent of the quantity of overlapping detections.

The outputs of the two branches are combined in a final computation step by subtracting the occupied area from the total interior area.

3.1.3 Branch C: OCR-Based Text Detection (Text Area Estimation)

The third branch uses an Optical Character Recognition (OCR) framework to detect the pixel area occupied by text. As seen in the examples below in Figure 4.1 Interior Area in Image 1, the text is not included in the free space detection, despite it not being a real-world object that would occupy the space. Therefore, the text must be detected, and the pixels of text counted, so that they can be added back to the interior area of the floor plan to make the final calculation result more accurate.

The specific type of OCR employed in this branch is EasyOCR framework, which is a pre-trained OCR framework built upon two different models. One is CRAFT (Character Region Awareness for Text Detection), which detects what is text. The other is CRNN (Convolutional Recurrent Neural Network), which reads what the text says. We will not be needing text recognition for anything, so we will only be using CRAFTs the text detection. The CRAFT architecture identifies text by estimating character-level regions and the affinity between neighboring characters (Baek et al., 2019). Unlike conventional bounding-box-based text detectors that estimate text using rectangular regions (Ren et al., 2015), CRAFT performs character-level localization and is therefore more capable of detecting irregularly shaped text (Zhou et al., 2017).

In the implementation, all the floor plan images are first loaded and processed sequentially. Each image is passed into EasyOCR's readtext function to generate raw OCR detections, which represent candidate text regions originally identified by the detector. In this step, several detection parameters are adjusted to improve sensitivity for small annotations, including `canvas_size = 3000`, `mag_ratio = 2.5`, `min_size = 2`, `text_threshold = 0.3`, `low_text = 0.15`, and `link_threshold = 0.15`. These settings allow smaller candidate text regions to be retained while the detector processes images at a higher scale. The model therefore returns the detected text regions and corresponding text predictions and confidence scores. After the raw OCR detections are generated, several filtering operations are applied to remove false positive detections. Candidate regions will be removed if their confidence score is lower than 0.25. Also, empty detections and excessively long text strings are removed since they are unlikely to be normal floor plan labels. Geometric filtering is then applied to remove the regions that are too small, too large, or too thin to suppress the noise from all fragments, symbols, and line intersections by calculating the width, height, bounding area, and top-edge angle of each detected region. Additional filtering is applied to regions with highly vertical aspect ratio or large tilted angle to avoid short wall segments, corners, and symbols to be misclassified as individual text characters.

The retained OCR detections are then converted into a binary text mask, which serves as the basis for pixel-level calculation. For each valid detected text region, the four corner points are first converted into integer pixel coordinates and then filled using OpenCV's polygon filling operation. Pixels inside the detected text regions are assigned a foreground value, while all other pixels remain background. The total text area is calculated by summing all foreground pixels in this mask. For visual verification, the generated text mask is overlaid on the original floor plan image, with detected text regions highlighted. Therefore, the output of Branch C consists of highlighted text masks together with a pixel-level estimation of the area occupied by text.

3.1.4 The Implementation Environment

PyTorch was used as the main deep learning framework in the Python 3 implementation of the system, while OpenCV was used for all image processing tasks in Branch A, B, and C. The Torchvision package has been utilized to instantiate and optimize the Mask R-CNN architecture. All model training and inference were carried out on Google Colaboratory's GPU-enabled environment.

4. Results and Analysis

4.1 Experimental Setup

All experiments were carried out on Google Colaboratory, with an NVIDIA GPU runtime. The Mask R-CNN model was loaded from a pre-trained checkpoint (maskrcnn_telstra.pth) and fine-tuned using a customized floor plan dataset annotated in COCO format, which included Train, Validation, and Test splits kept on Google Drive. The Torchvision package and PyTorch were used for all inference, and the model was set up with seven classes (background plus six object categories). OpenCV was used to implement the image processing branch (Bradski, 2000). Low-certainty detections were filtered for all inference runs using a confidence score threshold of 0.8, and duplicate proposals were eliminated using Non-Maximum Suppression (NMS) using an IoU threshold of 0.3. Four test images with no annotations were used to evaluate the pipeline.

4.2 Interior Area Extraction Results (Branch A)

Each floor plan image was processed by the image processing branch utilizing an OpenCV sequential pipeline. In order to isolate wall pixels as white foreground regions, each image was first converted to grayscale using `cv2.cvtColor` and then binary inverse thresholding at intensity value 200 using `cv2.threshold` with the `THRESH_BINARY_INV` flag. Next, a 5x5 kernel with two iterations was used to apply morphological closing using `cv2.morphologyEx` with `MORPH_CLOSE`, then dilation using a 7x7 kernel with two `cv2.iterations.dilate`, closing off wall border discontinuities (Gonzalez & Woods, 2018). CV2 was used to apply a flood fill operation seeded at coordinate (0, 0). The flood filling method used to eliminate all external pixels, leaving only the interior. The remaining foreground pixels were added together using `np.sum (flood == 255)` to determine the overall interior area, the green area in Figure 4.1 and Figure 4.2, and Figure 4.3 are the total interior area. Table 4.1 summarizes the results across the four test images.

Table 4.1 Results for Interior Area

Images	Interior Pixels
Image 1	69,486
Image 2	81,644
Image 3	97,888



Figure 4.1 Interior Area of Image 1



Figure 4.2 Interior Area of Image 2



Figure 4.3 Interior Area of Image 3

4.3 Object Detection Results (Branch B)

Each test picture was subjected to the predict function of the Mask R-CNN model. Bounding boxes, class labels, confidence scores, and binary masks were returned for each identified instance after the model used `T.ToTensor()` to take each image as a normalized tensor and `torch.no_grad()` to feed it through the network in evaluation mode. Only detections with confidence greater than 0.8 were kept after score filtering, and duplicate detections were suppressed by applying NMS with an IoU threshold of 0.3 using `torchvision.ops.nms`. Every class that wasn't background (`label > 0`) was kept. The detection results is shown in Figure 4.4, Figure 4.5, and Figure 4.6.



Figure 4.4 Object Detection in Image 1

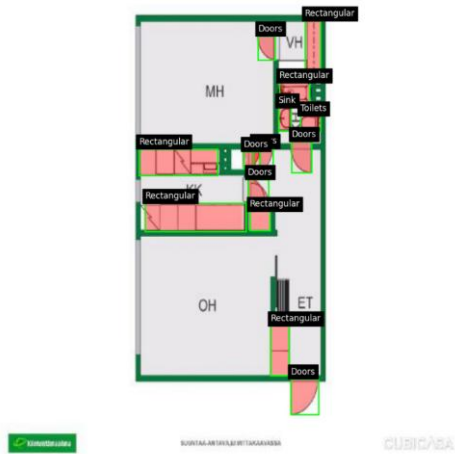


Figure 4.5 Object Detection in Image 2

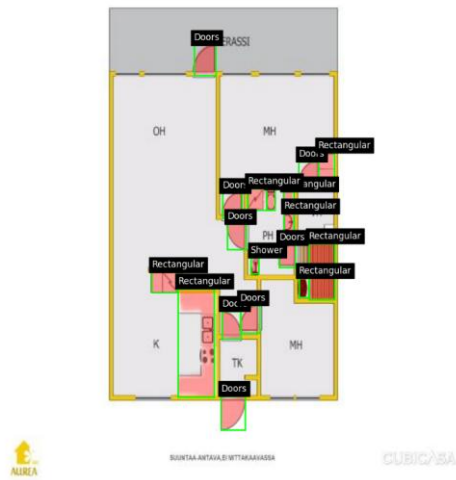


Figure 4.6 Object Detection in Image 3

4.4 Text Detection Results (Branch C)

The test images were tested using the EasyOCR model. According to the results, the detection branch successfully detected almost all the room labels and textual annotations in the floor plan images. As seen in Figures 4.7, 4.8 and 4.9, the detected text regions are highlighted in red overlays on top of the original floor plans. The filtering operations also worked

effectively to suppressed most false detections while retaining meaningful texts. Overall, the OCR text detection branch produced highlighted text masks together with a pixel-level estimation of the area occupied by text as expected. The number of pixels for text regions in different images is shown in Table 4.2.

Table 4.2 Results for Text Detection

Images	Text Pixels
Image 1	32169
Image 2	3706
Image 3	2901



Figure 4.7 Text detection in Image 1

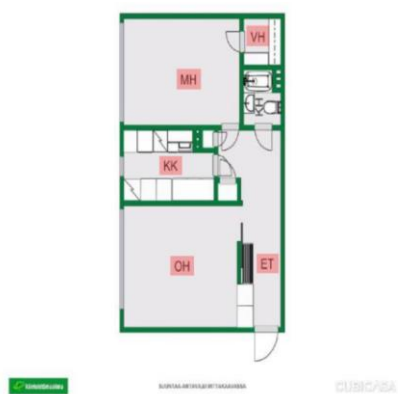


Figure 4.8 Text Detection in Image 2

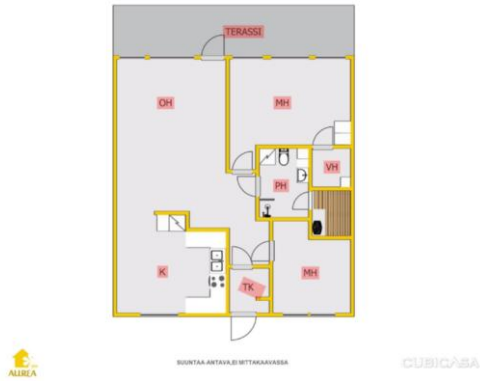


Figure 4.9 Text Detection in Image 3

4.5 Free Space Results Comparison

The `run_full_pipeline` function merged the results of both branches by removing the union of all anticipated object masks from the inner area mask, as shown below:
 $\text{Interior Area} - \text{Occupied Area} = \text{Free Area}$.

The occupied area was calculated by taking the logical union of all expected binary masks across all detections with `np.logical_or`, guaranteeing that each pixel was counted only once, regardless of how many objects overlapped at that position. The full free space results for all four test images are shown in Table 4.3.

Table 4.3 Free Space Results without Text Pixels

Image	Interior (px)	Occupied (px)	Free (px)	Free (%)	Occupied (%)
Image 1	69,486	14,756	54,730	78.7	21.24
Image 2	81,644	12,976	68,668	84.11	15.89
Image 3	97,888	10,133	87,755	89.65	10.35

To improve the accuracy of free space calculation, implementing text detection on the same sample images and then adding the text pixels to free space to get the following Table 4.4 as a comparison of Table 4.3:

Table 1.4 Free Space Results with Text Pixels

Image	Interior (px)	Occupied (px)	Free + Text (px)	Free (%)	Occupied (%)
Image 1	69,486	14,756	54,730 + 32169 = 86899	85.48	14.52

Image 2	81,644	12,976	68,668 + 3706 = 72374	84.8 0	15.2
Image 3	97,888	10,133	87,755 + 2901 = 90656	89.9 5	10.05

According to the comparison of Table 4.3 Free Space Results without Text Pixels and Table 4.4 Free Space Results with Text Pixels, we can see that the free space percentage shows a significant increase in Image 1, while in Image 2 and Image 3, the free space percentage also show slight increase. The extent of increase in free space percentage varies for each image, and it largely depends on how many pixels are present in each floor plan image.

4.6 Quantitative Evaluation

Table 4.5 below shows the complete set of quantitative parameters for each of the three test images. The predicted interior mask was compared to a ground-truth mask in order to calculate metrics.

Table 4.5 Quantitative Metrics

Image	IoU	Precision	Recall	F1-Score
Image 1	0.6327	0.6716	0.9162	0.7750
Image 2	0.7495	0.8333	0.8818	0.8568
Image 3	0.7987	0.8679	0.9092	0.8881

IoU calculates the ratio of overlap to union between the ground-truth and anticipated interior masks; a perfect match is represented by a score of 1.0. The IoU values for Images 1, 2 and 3 were 0.6327, 0.7495, and 0.7987, respectively, demonstrating a high degree of agreement between the true and anticipated interior regions. For a segmentation pipeline that just relies on image processing and does not employ a learned spatial model for interior identification, these values are regarded as robust. All four images had pairwise IoU of 0.0000, indicating that NMS with an IoU threshold of 0.3 effectively removed all duplicate detections. Zero pairwise overlap indicates that there are no superfluous pixels in the union mask and that the occupied area subtraction is mathematically clean, which is especially crucial for the accuracy of area computation.

The percentage of predicted interior pixels that are actually interior is reflected in precision. The precision of 0.6716 for Image 1 shows that a moderate fraction of pixels that the algorithm classified as interior were actually wall or outside pixels. This is typical of over-segmentation: the algorithm captures more area than actually exists. With precision scores of 0.8333 and 0.8679, respectively, images 2 and 3 demonstrated that the great majority of detected inner pixels were properly identified. Recall measures the proportion of true interior pixels that were successfully detected. Notably, images 1, 2, and 3 achieve recall scores of 0.9162, 0.8818, and

0.9092 respectively, demonstrating that the pipeline detects most of the true interior while maintaining high precision.

The F1-score is the harmonic mean of precision and recall, resulting in a single balanced measure of segmentation quality. Images 1, 2, and 3 obtained F1-scores of 0.7750, 0.8568, and 0.8881, demonstrating reliable and balanced interior identification.

5. Discussion

5.1 Interpretation of Image 1's Performance

This chapter critically assesses the experimental results reported in Chapter 4. It evaluates the design decisions made across the pipeline and finds the reasons that produced performance disparities across the three test images.

According to the table 4.2 in chapter 4, there are two things that instantly come to mind. First, recall ranges from 0.8818 to 0.9162 and is consistently high for all three images. This means that regardless of the type of floor layout, the flood-fill method consistently captures the great majority of true interior pixels. Second, precision rather than recall is the main cause of performance variance between images. The precision of Image 1 (0.6716) is significantly less than that of Images 2 and 3 (0.8333 and 0.8679), suggesting that although the flood fill detects the majority of the actual interior, it also incorporates a significant percentage of non-interior pixels in its prediction. Image 1's lower IoU of 0.6327 and F1-score of 0.7750 in comparison to Images 2 and 3's higher scores are caused by this precision difference.

With an IoU of 0.6327 and an F1-score of 0.7750, Image 1 performs functionally but noticeably worse than the other two images. The flood fill effectively locates interior space, as evidenced by the recall of 0.9162, which is actually the highest of the three photos. Precision is the limiting factor at 0.6716, which means that about one in three pixels that the algorithm labels as interior do not actually belong to the interior.

The performance difference between Images 2 and 3 and Image 1's F1-score of 0.7750 is significant but not disastrous. The score is significantly higher than a random baseline, indicating that the morphological operations and flood fill are functioning as intended; they just operate under less favorable structural conditions for this specific type of floor plan. The algorithm still accurately identifies interior space with a reasonable degree of accuracy.

5.2 Limitations of the Pipeline

- **Boundary Closure Sensitivity:** The external wall must create a closed perimeter around the design in order for the flood-fill approach to work. Without this structure, floor designs like Image 1 result in over-segmented interior masks that are less accurate. The accuracy of area estimation is impacted by the precision loss, even though Image 1 still earns an F1-score of 0.7750, suggesting the method has not catastrophically failed
- **Fixed Grayscale Threshold:** The `compute_interior_area` threshold value of 240 is not sensitive to wall color, contrast, or picture brightness. Wall pixels in floor layouts with mid-tone, colored, or low-contrast walls may be partially misclassified. While the morphological processes provide some robustness against this, the threshold remains a vulnerable point when applied to varied floor plan designs.

- Sensitivity of Morphological Parameters: sensitivity of morphological parameters. The kernel widths and iteration counts for dilation (7x7, two iterations) and morphological closing (5x5, two iterations) were chosen and might not generalize as well to floor designs with noticeably different line weights or wall thicknesses. Larger kernels cover gaps more aggressively, but they can erode narrow corridors; smaller kernels may create holes where flood fill can escape.

5.3 Recommendations for Improvement

The implementation of an automatic external boundary detecting phase prior to doing interior extraction would be the most significant enhancement. There are a number of practical methods for this. The outermost contour of each wall pixel can be isolated at the conventional image processing level using OpenCV's `cv2.findContours` using the `RETR_EXTERNAL` retrieval option, followed by `cv2.BoundPolyDP`. Before applying flood fill, a closed polygon can be fitted around the plan footprint using `convexHull` or `cv2.approxPolyDP` to fill in any gaps (Bradski & Kaehler, 2008). Based on the alpha-shape formulation presented by Edelsbrunner et al. (1983), a concave hull (alpha-shape) calculated using the `alphashape` Python module provides a closer fit to non-convex floor plan outlines for more complex plan forms where a convex hull would over-approximate the boundary. A lightweight semantic segmentation model, like a U-Net (Ronneberger et al., 2015) with a MobileNetV2 (Sandler et al., 2018) or EfficientNet-B0 (Tan & Le, 2019) encoder, could be trained at the deep learning level to directly predict a binary plan-footprint mask from the raw floor plan image, completely avoiding the need for contour detection. Regardless of the existence of a sketched outer wall, this footprint mask would thus act as a hard constraint on the flood fill, guaranteeing that it cannot extend beyond the actual design boundary.

Replacing a fixed value grayscale threshold of 240 with an adaptive technique would increase resistance to variations in wall color, image brightness, and print quality. The simplest upgrade is Otsu's thresholding (`cv2.threshold` with the `cv2.THRESH_OTSU` flag), which eliminates all reliance on the manually selected value of 240 by automatically choosing the ideal global threshold. For floor plans with non-uniform lighting or locally varying wall contrast, which are common in scanned or photographed documents, Gaussian adaptive thresholding (`cv2.adaptiveThreshold` with `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`) computes a separate threshold for each local neighbourhood of pixels, making it far more robust to gradients in image brightness across the plan. Additionally, regarding the sensitivity of morphological parameters, there are various ways to deal with this. First, by analyzing the width distribution of dark-pixel runs in the binarized image using a run-length encoding approach, a method used in document image analysis to characterize stroke widths, automatic kernel size estimation based on the dominant wall thickness in each image could be implemented (Dodge et al., 2017). The morphological stage would then be adaptable to each input image rather than fixed since the kernel size would be directly informed by the projected wall thickness.

For the OCR text detection model, although the branch was able to identify most regular horizontal texts, its performance was still limited when handling vertically arranged, tilted, or irregularly arranged text. This is because the current implementation mainly relies on the default EasyOCR detection output and post-processing filters, which are more effective for standard text regions. In future work, the text detection branch can be improved by introducing rotation-aware preprocessing before OCR inference, where the input images can be automatically rotated

through multiple angles, such as 90° , 180° , and 270° , before applying OCR detection separately on each rotated image. The detected text regions could then be transformed back to the original image coordinates and merged into a unified text mask. This approach may improve the localization of vertically arranged texts and irregular text regions that are not successfully detected in the current pipeline.

An alternative way of structuring the final pipeline would be to do it like in Figure 5.1 In this version branch C will also change. Instead of extracting the number pixels of text to add back in the next step, the boxes that are detected will be covered in a box that matches the colour of the surrounding area. A copy of the original image is then created with the area inside the detection box is filled in with that colour, thus removing the text. It is this copy that is analysed in Branch A (interior area extraction) instead of the original image. This means that the next step only requires removing the extracted area of the objects that Branch B has detected with Mask R-CNN. An example of this is shown in Figure 5.2 The boxes were filled in with the average colour detected in the pixels surrounding the box up to 6 pixels away.

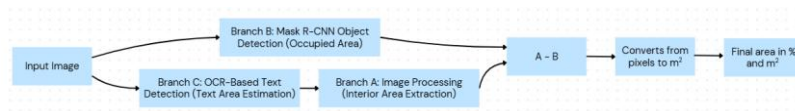


Figure 5.1 Alternative Pipeline

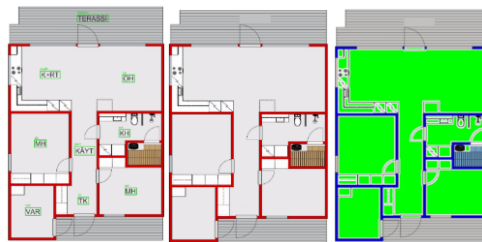


Figure 5.2 Alternative Branch C

This alternative method might have been an improvement since the interior area extraction might not have missed every single pixel of text, and the text pixel extraction might not have gotten every single pixel of text. So the sum of the two might not be the exact same answer as if there was no text. It also removes the need to detect which text areas are outside of the walls, since the area extraction code only looks at what is inside of the walls anyway.

6. Conclusion

This project developed and evaluated to quantify interior space, occupied area, and free usable space without the need for manual measurement. The study's primary finding is that, when applied to floor plans with clearly defined exterior boundaries, the pipeline performs consistently across all three branches; however, in Branch A, wall boundary ambiguity specifically limits performance rather than any systematic flaws in the deep learning or OCR components. Future research should concentrate on adaptive thresholding, morphological parameter estimation based on wall thickness, and automatic exterior boundary detection and closure. These approaches

should improve performance on ambiguous floor plans and match the excellent outcomes obtained on well-formatted inputs. When combined, the three branches provide a useful, expandable framework for automated floor plan spatial analysis that can be applied to actual equipment planning and facility management.

Bibliography

- Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character region awareness for text detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9365–9374.
- Bradski, G. (2000). The OpenCV library. *Dr. Dobbs's Journal of Software Tools*, 25, 120–125.
- Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 233–240.
- Dodge, S., Xu, J., & Stenger, B. (2017).** Parsing floor plan images. In *Proceedings of the 15th IAPR International Conference on Machine Vision Applications* (pp. 358–361). IEEE.
- Duda, R. O., & Hart, P. E. (1972). Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1), 11–15. <https://doi.org/10.1145/361237.361242>
- Edelsbrunner, H., Kirkpatrick, D., & Seidel, R. (1983). On the shape of a set of points in the plane. *IEEE Transactions on Information Theory*, 29(4), 551–559. <https://doi.org/10.1109/TIT.1983.1056714>
- Girshick, R. (2015). Fast R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 1440–1448)
- Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 580–587).
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.
- He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 2961–2969). IEEE.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). IEEE
- Iakubovskii, P. (2019). *Segmentation models PyTorch* [Software]. GitHub. https://github.com/qubvel/segmentation_models_pytorch
- Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2117–2125). IEEE
- Marcel, S., & Rodriguez, Y. (2010). Torchvision the machine-vision package of Torch. In *Proceedings of the ACM International Conference on Multimedia* (pp. 1485–1488). ACM.
- Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66. <https://doi.org/10.1109/TSMC.1979.4310076>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* (Vol. 32, pp. 8026–8037). Curran Associates.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28.

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention – MICCAI 2015* (pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). <https://doi.org/10.1109/CVPR.2018.00474>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning* (pp. 6105–6114). PMLR.

Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., & Liang, J. (2017). EAST: An efficient and accurate scene text detector. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 5551–5560.

Acknowledgements

The authors would like to sincerely thank the academic staff and supervisors at the University of Melbourne for their assistance, advice, and feedback during this capstone project. The authors additionally acknowledges the usage of Google Colaboratory for the GPU processing environment required for training the Mask R-CNN model, as well as Google Drive for dataset storage and maintenance during the research. The study's floor plan photos were from publicly accessible online sources.